

PZ-OV2640 摄像头模块开发手册

本手册将教大家如何在天马&麒麟开发板上使用 PZ-OV2640 摄像头模块。本章分为如下几部分内容：

- 1 PZ-OV2640 模块简介
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-OV2640 模块简介

1.1 特性参数

PZ-OV2640 是深圳普中科技推出的一款 200W 像素高清摄像头模块。该模块采用了一颗 OV2640 集成化模组,从而使模块更加精致小巧,而且该模组还可重复利用。PZ-OV2640 模块采用该 OV2640 模组作为核心部件,集成有源晶振和 LDO,接口简单,使用非常方便。

PZ-OV2640 模块的特点如下:

- 高灵敏度、低电压适合嵌入式应用
- 标准的 SCCB 接口,兼容 IIC 接口
- 支持 RawRGB、RGB(RGB565/RGB555)、GRB422、YUV(422/420)和 YCbCr(422)

输出格式

- 支持 UXGA、SXGA、SVGA 以及按比例缩小到从 SXGA 到 40*30 的任何尺寸
- 支持自动曝光控制、自动增益控制、自动白平衡、自动消除灯光条纹、自动黑电平校准等自动控制功能。同时支持色饱和度、色相、伽马、锐度等设置。
- 支持图像缩放、平移和窗口设置
- 支持图像压缩,即可输出 JPEG 图像数据
- 自带嵌入式微处理器
- 集成有源晶振,无需外部提供时钟
- 集成 LDO,仅需提供 3.3V 电源即可正常工作

PZ-OV2640 模块各项参数如下图所示。

项目	说明
接口类型	数据接口：8 位数据 控制接口：SCCB（类 IIC 协议）
输出格式	RawRGB、RGB(RGB565/RGB555)、GRB422、YUV(422/420)、YCbCr(422)、JPEG 数据
输出位宽	8 位
输出像素	UXGA(1600*1200)及以下到 40*30 的任意尺寸
最大帧率	UXGA(1600*1200)@15 帧 SVGA(800*600)@30 帧 CIF(352*288)@60 帧
灵敏度	0.6V/Lux-sec
信噪比	40dB
动态范围	50dB
镜头光圈	F2.0
镜头视角	62°
镜头焦距	可调
工作温度	-30℃~70℃
模块尺寸	24mm*32.98mm
电源电压	3.3V
IO 口电平 ^①	2.8V LVTTTL，可兼容 3.3V
功耗	40mA

注①：模块 IO 电压是 2.8V，不过对于 3.3V 系统，是可以直接兼容的。所以 3.3V 的 MCU 无需任何处理，直接连接模块即可。不过如果是 5V 的 MCU，建议在信号线上串接 1K 左右电阻，做限流处理。

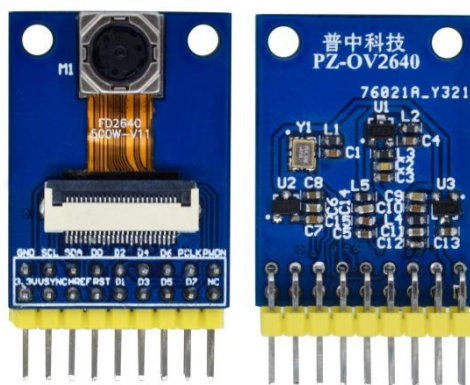
UXGA，即分辨率为 1600*1200 的输出格式，类似的还有：SXGA(1280*1024)、WXGA+(1440*900)、XVGA(1280*960)、WXGA(1280*800)、XGA(1024*768)、SVGA(800*600)、VGA(640*480)、CIF(352*288)、WQVGA(400*240)、QCIF(176*144)和 QQVGA(160*120)等。

1.2 使用说明

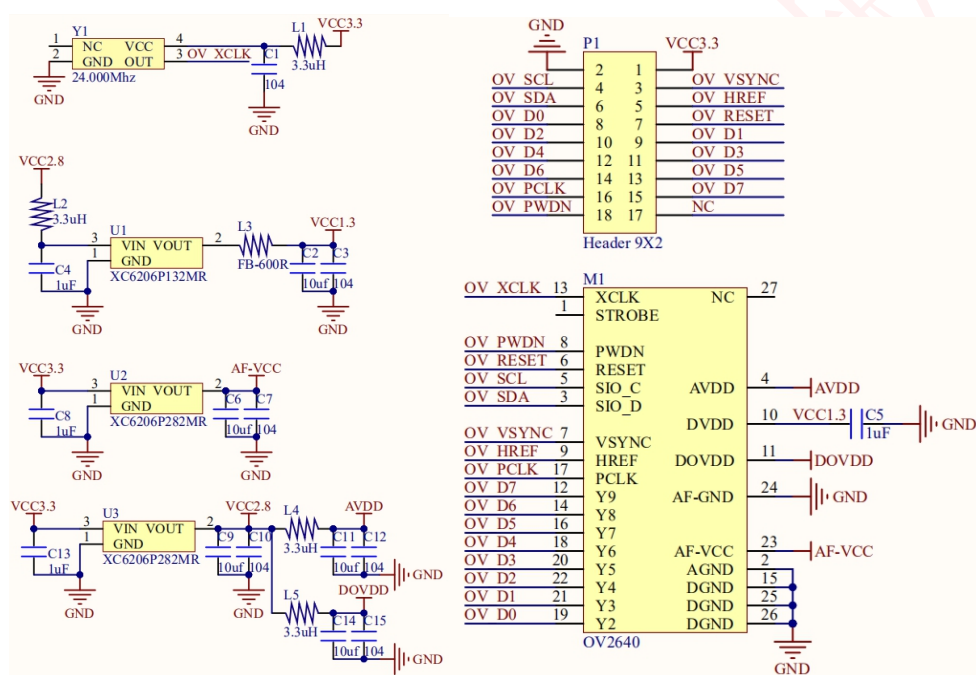
1.2.1 模块引脚说明

PZ-0V2640 摄像头模块通过 2*9 的弯排针（2.54mm 间距）同外部连接，模块可以与天马&麒麟等开发板直接对接，而普中 F407 最小系统开发板则可以通过杜邦线连接模块进行测试。所有普中 STM32F407 系列开发板都可使用该例程，用户可以直接在这些开发板上，对模块进行测试。

PZ-0V2640 摄像头模块外观如下图所示：



PZ-OV2640 摄像头模块原理图如下图所示：



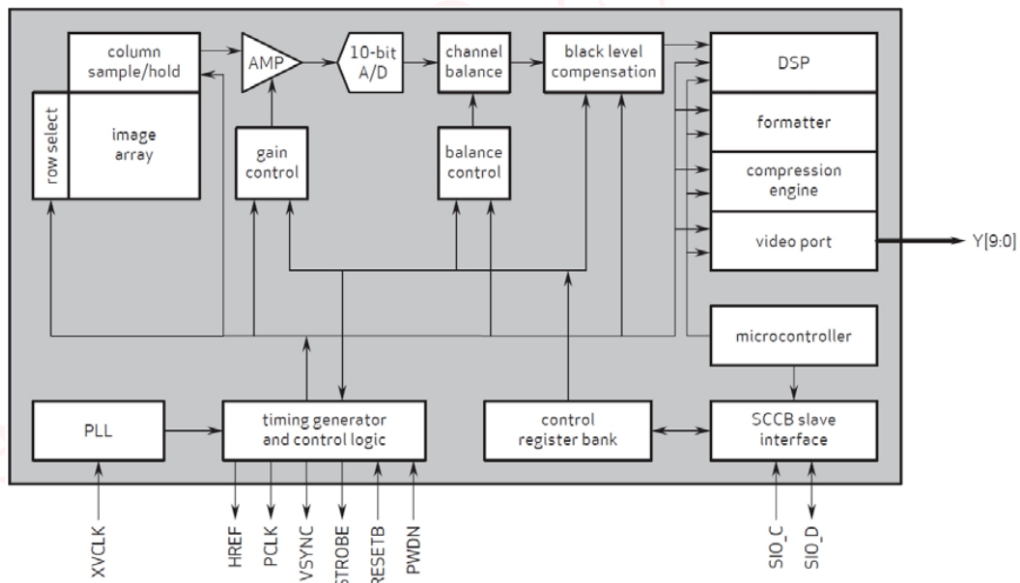
从图中可以看出，模块自带了 1.3V 和 2.8V 的稳压芯片，给 OV2640 供电，因此外部仅需提供 3.3V 电压即可；同时自带了一个 24M 的有源晶振，所以模块不需要外部提供时钟。PZ-OV2640 摄像头模块通过一个 2*9 的排针（P1）同外部电路连接，各引脚的详细描述如下图所示：

序号	名称	说明
1	VCC	3.3V 电源输入脚
2	GND	地线
3	OV_VSYNC	帧同步信号 (OUT ^②)
4	OV_SCL	SCCB 时钟线 (IN ^①)
5	OV_HREF	行参考信号 (OUT)
6	OV_SDA	SCCB 数据线 (IN/OUT)
7	OV_RESET	复位信号 (低电平有效) (IN)
8-15	OV_D0~D7	数据线 (OUT)
16	OV_PCLK	像素时钟 (OUT)
17	NC	未用到
18	OV_PWDN	掉电模式使能 (高电平有效) (IN)

注①、②：IN，表示输入信号；OUT 表示输出信号；IN/OUT 表示即可以输出，也可以做输入信号。从上图可以看出，PZ-OV2640 摄像头模块，总共需要 15 个 IO 口驱动。

1.2.2 OV2640 内部功能结构介绍

OV2640 的功能框图如下图所示：



OV2640 传感器包括如下一些功能模块。

(1) 感光阵列 (Image Array)

OV2640 总共有 1632*1232 个像素，最大输出尺寸为 UXGA (1600*1200)，即 200W 像素。

(2) 模拟信号处理 (Analog Processing)

模拟信号处理所有模拟功能，并包括：模拟放大 (AMP)、增益控制、通道

平衡和平衡控制等。

(3) 10 位 A/D 转换 (A/D)

原始的信号经过模拟放大后，分 G 和 BR 两路进入一个 10 位的 A/D 转换器，A/D 转换器工作频率高达 20M，与像素频率完全同步（转换的频率和帧率有关）。除 A/D 转换器外，该模块还有黑电平校正（BLC）功能。

(4) 数字信号处理器 (DSP)

这个部分控制由原始信号插值到 RGB 信号的过程，并控制一些图像质量：

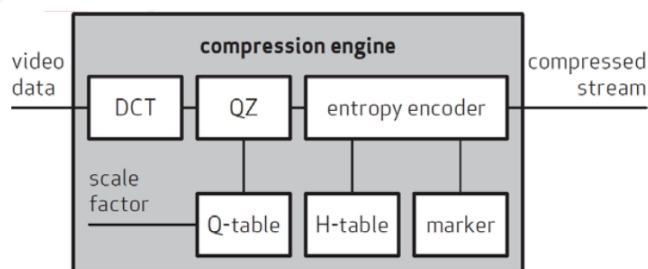
- 边缘锐化（二维高通滤波器）
- 颜色空间转换（原始信号到 RGB 或者 YUV/YCbYCr）
- RGB 色彩矩阵以消除串扰
- 色相和饱和度的控制
- 黑/白点补偿
- 降噪
- 镜头补偿
- 可编程的伽玛
- 十位到八位数据转换

(5) 输出格式模块 (Output Formatter)

该模块按设定优先级控制图像的所有输出数据及其格式。

(6) 压缩引擎 (Compression Engine)

压缩引擎框图如下图所示：



从上图可以看出，压缩引擎主要包括三部分：DCT、QZ 和 entropy encoder（熵编码器），将原始的数据流，压缩成 jpeg 数据输出。

(7) 微处理器 (Microcontroller)

OV2640 自带了一个 8 位微处理器，该处理器有 512 字节 SRAM，4KB 的 ROM，它提供一个灵活的主机到控制系统的指令接口，同时也具有细调图像质量

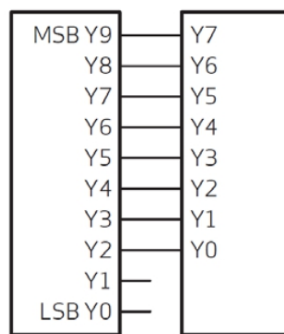
的功能。

(8) SCCB 接口 (SCCB Interface)

SCCB 接口控制图像传感器芯片的运行，详细使用方法参照模块参考资料的《OmniVision Technologies Serial Camera Control Bus (SCCB) Specification》这个文档。

(9) 数字视频接口 (Digital Video Port)

OV2640 拥有一个 10 位数字视频接口(支持 8 位接法)，其 MSB 和 LSB 可以程序设置先后顺序，PZ-OV2640 模块采用默认的 8 位连接方式，如下图所示：



1.2.2 OV2640 传感器窗口介绍

接下来，我们介绍一下 OV2640 的传感器窗口设置、图像尺寸设置、图像窗口设置和图像输出大小设置，这几个设置与我们的正常使用密切相关，有必要了解一下。其中，除了传感器窗口设置是直接针对传感器阵列的设置，其他都是 DSP 部分的设置了，接下来我们一个个介绍。

传感器窗口设置，该功能允许用户设置整个传感器区域（1632*1220）的兴趣部分，也就是在传感器里面开窗，开窗范围从 2*2~1632*1220 都可以设置，不过要求这个窗口必须大于等于随后设置的图像尺寸。传感器窗口设置，通过：0X03/0X19/0X1A/0X07/0X17/0X18 等寄存器设置，寄存器定义请看模块参考资料 OV2640_DS(1.6).pdf 这个文档（下同）。

图像尺寸设置，也就是 DSP 输出（最终输出到 LCD 的）图像的最大尺寸，该尺寸要小于等于前面我们传感器窗口设置所设定的窗口尺寸。图像尺寸通过：0XC0/0XC1/0X8C 等寄存器设置。

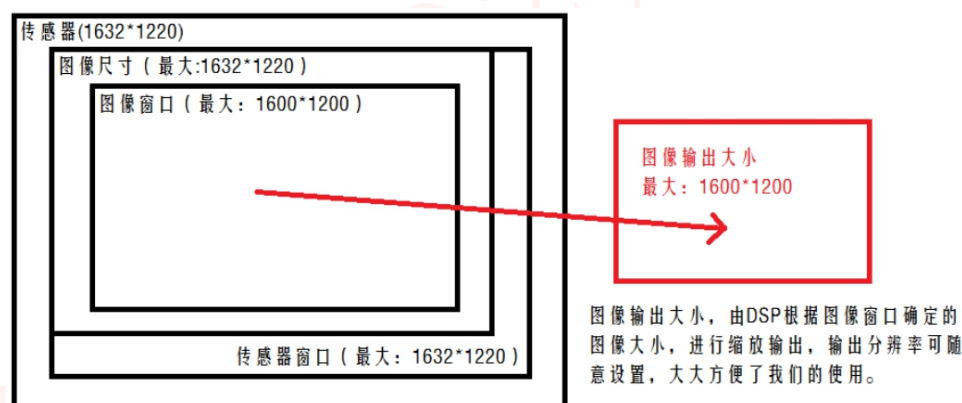
图像窗口设置，这里起始和前面的传感器窗口设置类似，只是这个窗口是在

我们前面设置的图像尺寸里面，再一次设置窗口大小，该窗口必须小于等于前面设置的图像尺寸。该窗口设置后的图像范围，将用于输出到外部。图像窗口设置通过：0X51/0X52/0X53/0X54/0X55/0X57 等寄存器设置。

图像输出大小设置，这是最终输出到外部的图像尺寸。该设置将图像窗口设置所决定的窗口大小，通过内部 DSP 处理，缩放成我们输出到外部的图像大小。该设置将会对图像进行缩放处理，如果设置的图像输出大小不等于图像窗口设置图像大小，那么图像就会被缩放处理，只有这两者设置一样大的时候，输出比例才是 1:1 的。

因为 OmniVision 公司公开的文档，对这些设置实在是没有详细介绍。只能从他们提供的初始化代码（还得去 linux 源码里面移植过来）里面去分析规律，所以，这几个设置，都是根据 OV2640 的调试经验，以及相关文档总结出来的，不保证百分百正确，如有错误，还请大家指正。

以上几个设置，光看文字可能不太清楚，这里我们画一个简图有助于大家理解，如下图所示：



上图，最终红色框所示的图像输出大小，才是 OV2640 输出给外部的图像尺寸，也就是显示在 LCD 上面的图像大小。当图像输出大小与图像窗口不等时，会进行缩放处理，在 LCD 上面看到的图像将会变形。

1.2.2 串行摄像头控制总线(SCCB)简介

PZ-OV2640 摄像头模块的所有配置，都是通过 SCCB 总线来实现的，SCCB 全称是：Seril Camera Control Bus 即串行摄像头控制总线，它由两条数据线组成：一个是用于传输时钟信号的 SIO_C（即 OV_SCL），另一个是用于传输数据

信号的 SIO_D (即 OV_SDA)。

SCCB 的传输协议与 IIC 协议极其相似,只不过 IIC 在每传输完一个字节后,接收数据的一方要发送一位的确认数据,而 SCCB 一次要传输 9 位数据,前 8 位为有用数据,而第 9 位数据在写周期中是 don't care 位(即不必关心位),在读周期中是 NA 位。SCCB 定义数据传输的基本单元为相(phase),即一个相传输一个字节数据。

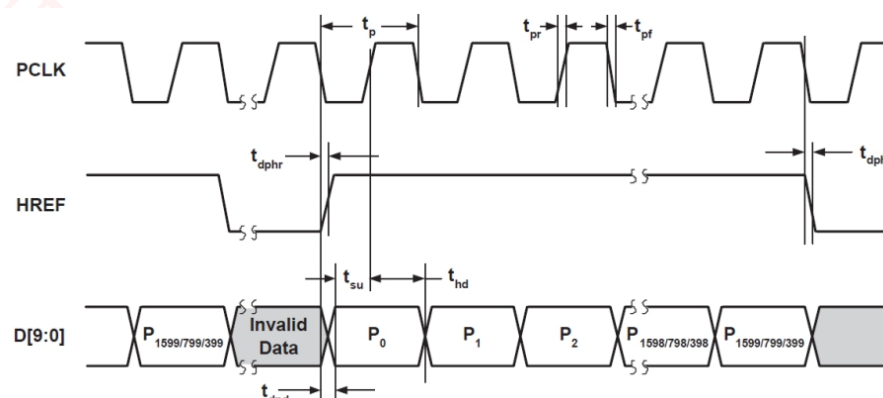
SCCB 只包括三种传输周期,即 3 相写传输周期(三个相依次为设备从地址,内存地址,所写数据),2 相写传输周期(两个相依次为设备从地址,内存地址)和 2 相读传输周期(两个相依次为设备从地址,所读数据)。当需要写操作时,应用 3 相写传输周期,当需要读操作时,依次应用 2 相写传输周期和 2 相读传输周期。

关于 SCCB 的详细介绍,请大家参考模块资料“\4--参考资料\《OmniVision Technologies Seril Camera Control Bus(SCCB) Specification.pdf》”这个文档。

OV2640 的初始化,需要配置大量寄存器,这里我们就不给大家多做介绍了,请大家参考模块资料“\4--参考资料\《OV2640 Software Application Notes 1.03.pdf》”这个文档。

1.2.3 行像素输出时序介绍

PZ-OV2640 的数据输出,是在行参考信号和像素时钟的控制下,有序输出的,默认的行像素输出时序,如下图所示:



当行参考信号 (HREF) 为高电平时,表示数据有效,此时,每输出一个像素

时钟（PCLK），就输出一个数据（8/10 位）。数据在 PCLK 的下降沿更新，所以，MCU 在 PCLK 的上升沿读取数据。

注意：图中的 t_p 表示像素周期，像素周期可能等于 1 个 PCLK，也可能等于 2 个 PCLK。在 RGB/YUV 输出格式下，每个 $t_p=2$ 个 Tpc1k，如果是 Raw RGB 格式，则一个 $t_p=1$ 个 Tpc1k，其中：Tpc1k 即 PCLK 信号的周期。

当使用 RGB565 输出的时候， $t_p=2$ 个 Tpc1k，每一个像素是 2 个字节，低字节在前，高字节在后，那么如果采用 UXGA 分辨率，那么每一行就有 1600×2 个 PCLK，用于输出行像素数据。

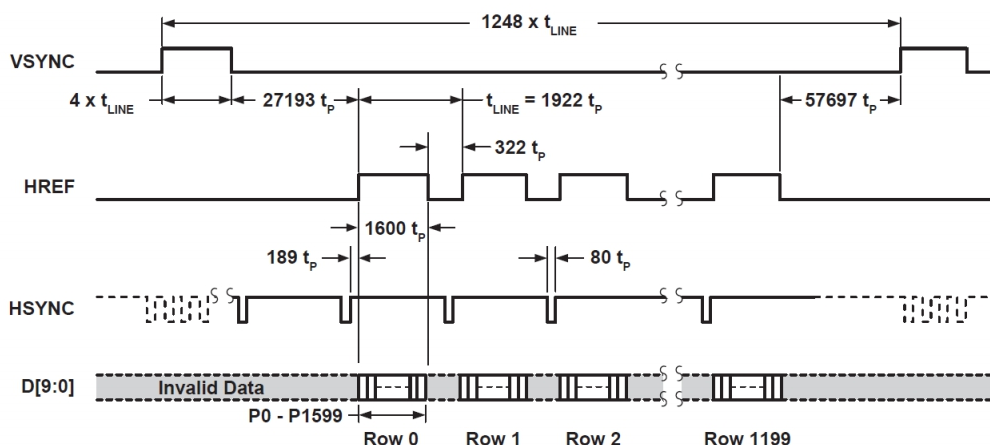
当使用 JPEG 输出的时候，输出的是压缩后的数据，这里与普通的行像素输出时序略有不同，普通的行像素输出，HREF 是连续的，也就是整个一行像素输出，HREF 会一直保持高电平，而 JPEG 输出的时候，HREF 则是不连续的，即一行数据输出时，HREF 可能会有很多次低电平出现。但是这并不影响我们读取数据，只需要判定 HREF 为高电平，再读取数据，就可以了。JPEG 输出时，不存在高低字节的概念，只需要从头到尾，将所有数据读取保存下来，就可以完成一次 JPEG 数据采集。

输出的 JPEG 数据以：0xFF,0xD8 开头，以 0xFF,0xD9 结尾，且在 0xFF,0xD8 之前，或者 0xFF,0xD9 之后，会有不定数量的其他数据存在（一般是 0），这些数据我们直接忽略即可，将得到的 0xFF,0xD8~0xFF,0xD9 之间的数据，保存为 .jpg/.jpeg 文件，就可以直接在电脑上打开看到图像了。

注意：PCLK 的频率可达 36Mhz，所以 MCU 在读取数据的时候，必须速度够快才可以，否则就可能出现数据丢失。对于速度不够快的 MCU，我们可以通过设置 0V2640 的寄存器（0XD3 和 0X11），设置 PCLK 和时钟的分频来降低 PCLK 速度，从而使得低速 MCU 也可以读取 0V2640 的数据。不过这样会降低帧率。

1.2.4 帧时序介绍

前面我们介绍了行像素输出时序，一帧图像数据，实际上就是由 N 个行像素输出时序组成的。这里我们以 UXGA 的帧时序为例进行介绍，UXGA 的帧时序如下图所示：



图中 t_{LINE} 即行输出时间， t_p 则是像素周期。VSYNC 是帧同步信号，每一个脉冲，表示一个新帧的开始，而整个帧周期内，有 1200 次行像素（Row）输出，每一行为 1600 个像素，这样得到的输出数据，刚好是 1600×1200 的分辨率，即 UXGA 的分辨率。

HSYNC 为行同步信号，用于同步行输出数据，不过，我们的 PZ-0V2640 模块，并没有引出该信号。大家可以用 HREF 做同步即可。

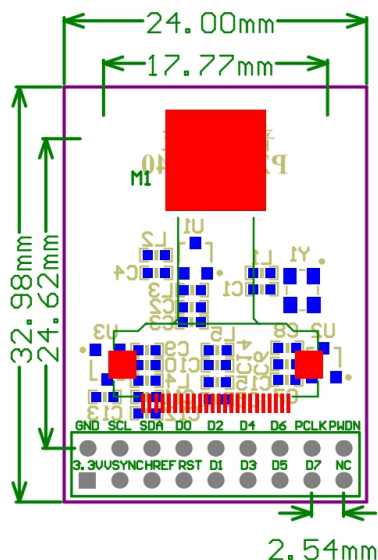
其他分辨率下，帧时序图，与 UXGA 大同小异，请大家参考模块资料“\4--参考资料\《0V2640_DS(1.6).pdf》”这个文档。

1.2.5 模块使用注意事项

PZ-0V2640 摄像头模块使用的是一体化的 0V2640 模组，本身就含有调焦镜头。当显示画面模糊时，可通过旋转模组镜头来调节清晰度。如果用手调节比较困难时，可以用镊子夹住镜头旋转。

1.3 结构尺寸

PZ-0V2640 摄像头模块尺寸结构如下图所示：



1.4 STM32F4 DCMI 接口简介

STM32F4 自带了一个数字摄像头 (DCMI) 接口, 该接口是一个同步并行接口, 能够接收外部 8 位、10 位、12 位或 14 位 CMOS 摄像头模块发出的高速数据流。可支持不同的数据格式: YCbCr4:2:2/RGB565 逐行视频和压缩数据 (JPEG)。

STM32F4 DCMI 接口特点:

- 8 位、10 位、12 位或 14 位并行接口
- 内嵌码/外部行同步和帧同步
- 连续模式或快照模式
- 裁剪功能
- 支持以下数据格式:
 - 1, 8/10/12/14 位逐行视频: 单色或原始拜尔 (Bayer) 格式
 - 2, YCbCr 4:2:2 逐行视频
 - 3, RGB 565 逐行视频
 - 4, 压缩数据: JPEG

DCMI 接口包括如下一些信号:

- 1, 数据输入 (D[0:13]), 用于接摄像头的的数据输出, 接 OV2640 我们只用了 8 位数据。
- 2, 水平同步 (行同步) 输入 (HSYNC), 用于接摄像头的 HSYNC/HREF 信号。
- 3, 垂直同步 (场同步) 输入 (VSYNC), 用于接摄像头的 VSYNC 信号。

4, 像素时钟输入 (PIXCLK), 用于接摄像头的 PCLK 信号。

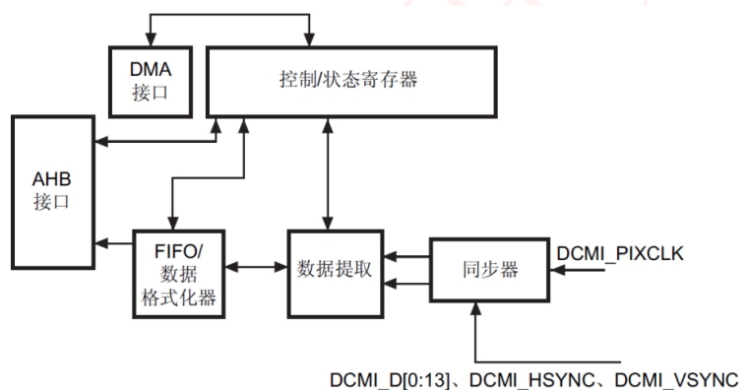
DCMI 接口是一个同步并行接口, 可接收高速 (可达 54 MB/s) 数据流。该接口包含多达 14 条数据线 (D13-D0) 和一条像素时钟线 (PIXCLK)。像素时钟的极性可以编程, 因此可以在像素时钟的上升沿或下降沿捕获数据。

DCMI 接收到的摄像头数据被放到一个 32 位数据寄存器 (DCMI_DR) 中, 然后通过通用 DMA 进行传输。图像缓冲区由 DMA 管理, 而不是由摄像头接口管理。

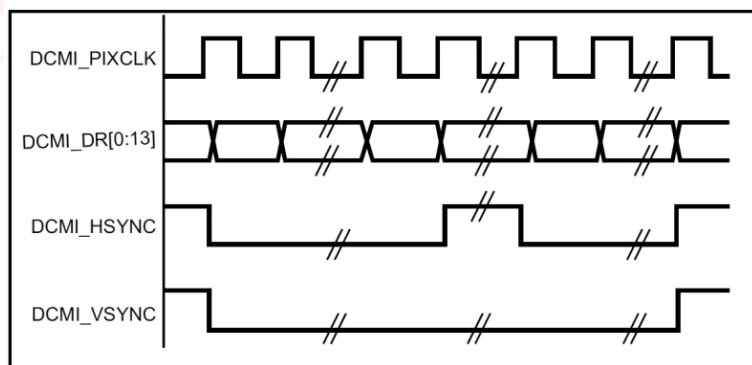
从摄像头接收的数据可以按行/帧来组织 (原始 YUV/RGB/拜尔模式), 也可以是一系列 JPEG 图像。要使能 JPEG 图像接收, 必须将 JPEG 位 (DCMI_CR 寄存器的位 3) 置 1。

数据流可由可选的 HSYNC (水平同步) 信号和 VSYNC (垂直同步) 信号硬件同步, 或者通过数据流中嵌入的同步码同步。

STM32F4 DCMI 接口的框图如下图所示:



DCMI 接口的数据与 PIXCLK (即 PCLK) 保持同步, 并根据像素时钟的极性在像素时钟上升沿/下降沿发生变化。HSYNC (HREF) 信号指示行的开始/结束, VSYNC 信号指示帧的开始/结束。DCMI 信号波形如下图所示:



上图中, 对应设置为: DCMI_PIXCLK 的捕获沿为下降沿, DCMI_HSYNC 和 DCMI_VSYNC 的有效状态为 1, 注意, 这里的有效状态实际上对应的是指示数据

在并行接口上无效时，HSYNC/VSYNC 引脚上面的引脚电平。

本实验我们用到 DCMI 的 8 位数据宽度，通过设置 DCMI_CR 中的 EDM[1:0]=00 设置。此时 DCMI_D0~D7 有效，DCMI_D8~D13 上的数据则忽略，这个时候，每次需要 4 个像素时钟来捕获一个 32 位数据。捕获的第一个数据存放在 32 位字的 LSB 位置，第四个数据存放在 32 位字的 MSB 位置，捕获数据字节在 32 位字中的排布如下图所示：

字节地址	31:24	23:16	15:8	7:0
0	D _{n+3} [7:0]	D _{n+2} [7:0]	D _{n+1} [7:0]	D _n [7:0]
4	D _{n+7} [7:0]	D _{n+6} [7:0]	D _{n+5} [7:0]	D _{n+4} [7:0]

从图中可以看出，STM32F4 的 DCMI 接口，接收的数据是低字节在前，高字节在后的，所以，要求摄像头输出数据也是低字节在前，高字节在后才可以，否则就还得程序上处理字节顺序，会比较麻烦。

DCMI 接口支持 DMA 传输，当 DCMI_CR 寄存器中的 CAPTURE 位置 1 时，激活 DMA 接口。摄像头接口每次在其寄存器中收到一个完整的 32 位数据块时，都将触发一个 DMA 请求。

DCMI 接口支持两种同步方式：内嵌码同步和硬件（HSYNC 和 VSYNC）同步。我们简单介绍下硬件同步，详细介绍可以参考开发板资料中《STM32F4xx 中文数据手册》第 13.5.3 节。

硬件同步模式下将使用两个同步信号（HSYNC/VSYNC）。根据摄像头模块/模式的不同，可能在水平/垂直同步期间内发送数据。由于系统会忽略 HSYNC/VSYNC 信号有效电平期间内接收的所有数据，HSYNC/VSYNC 信号相当于消隐信号。

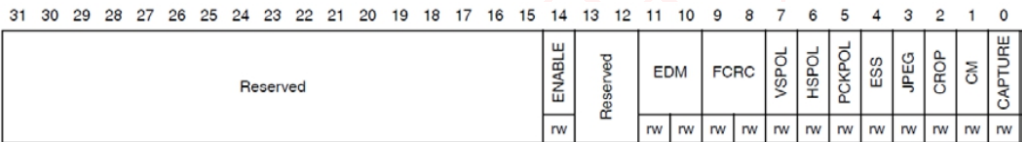
为了正确地将图像传输到 DMA/RAM 缓冲区，数据传输将与 VSYNC 信号同步。选择硬件同步模式并启用捕获（DCMI_CR 中的 CAPTURE 位置 1）时，数据传输将与 VSYNC 信号的无效电平同步（开始下一帧时）。之后传输便可以连续执行，由 DMA 将连续帧传输到多个连续的缓冲区或一个具有循环特性的缓冲区。为了允许 DMA 管理连续帧，每一帧结束时都将激活 VSIF（垂直同步中断标志，即帧中断），我们可以利用这个帧中断来判断是否有一帧数据采集完成，方便处理数据。

DCMI 接口的捕获模式支持：快照模式和连续采集模式。一般我们使用连续采集模式，通过 DCMI_CR 中的 CM 位设置。另外，DCMI 接口还支持实现了 4 个

字深度的 FIFO，配有一个简单的 FIFO 控制器，每次摄像头接口从 AHB 读取数据时读指针递增，每次摄像头接口向 FIFO 写入数据时写指针递增。因为没有溢出保护，如果数据传输率超过 AHB 接口能够承受的速率，FIFO 中的数据就会被覆盖。如果同步信号出错，或者 FIFO 发生溢出，FIFO 将复位，DCMI 接口将等待新的数据帧开始。

关于 DCMI 接口的其他特性，我们这里就不再介绍了，请大家参考开发板资料《STM32F4xx 中文参考手册》第 13 章相关内容。

本实验我们将 OV2640 默认配置为 UXGA 输出，也就是 1600*1200 的分辨率，输出信号设置为：VSYNC 高电平有效，HREF 高电平有效，输出数据在 PCLK 的下降沿输出（即上升沿的时候，MCU 才可以采集）。这样，STM32F4 的 DCMI 接口就必须设置为：VSYNC 低电平有效、HSYNC 低电平有效和 PIXCLK 上升沿有效，这些设置都是通过 DCMI_CR 寄存器控制的，该寄存器描述如下图所示：



ENABLE: 该位用于设置是否使能 DCMI，不过，在使能之前，必须将其他配置设置好。

FCRC[1:0]: 这两个位用于帧率控制，我们捕获所有帧，所以设置为 00 即可。

VSPOL: 该位用于设置垂直同步极性，也就是 VSYNC 引脚上面，数据无效时的电平状态，根据前面说所，我们应该设置为 0。

HSPOL: 该位用于设置水平同步极性，也就是 HSYNC 引脚上面，数据无效时的电平状态，同样应该设置为 0。

PCKPOL: 该位用于设置像素时钟极性，我们用上升沿捕获，所以设置为 1。

CM: 该位用于设置捕获模式，我们用连续采集模式，所以设置为 0 即可。

CAPTURE: 该位用于使能捕获，我们设置为 1。该位使能后，将激活 DMA，DCMI 等待第一帧开始，然后生成 DMA 请求将收到的数据传输到目标存储器中。

注意：该位必须在 DCMI 的其他配置（包括 DMA）都设置好了之后，才设置！！

DCMI_CR 寄存器的其他位，我们就不介绍了，另外 DCMI 的其他寄存器这里也不再介绍，请大家参考开发板中资料《STM32F4xx 中文参考手册》第 13.8 节。

1.5 STM32F4 DCMI 配置步骤

最后，我们来看下用 DCMI 驱动 OV2640 的步骤：

(1) 配置 OV2640 控制引脚，并配置 OV2640 工作模式。

在启动 DCMI 之前，我们先设置好 OV2640。OV2640 通过 OV_SCL 和 OV_SDA 进行寄存器配置，同时还有 OV_PWDN/OV_RESET 等信号，我们也需要配置对应 IO 状态，先设置 OV_PWDN=0，退出掉电模式，然后拉低 OV_RESET 复位 OV2640，之后再设置 OV_RESET 为 1，结束复位，然后就是对 OV2640 的大把寄存器进行配置了。然后，可以根据我们的需要，设置成 RGB565 输出模式，还是 JPEG 输出模式。

(2) 配置相关引脚的模式和复用功能（AF13），使能时钟。

OV2640 配置好之后，再设置 DCMI 接口与摄像头模块连接的 IO 口，使能 IO 和 DCMI 时钟，然后设置相关 IO 口为复用功能模式，复用功能选择 AF13(DCMI 复用)。

(3) 配置 DCMI 相关设置。

这一步，主要通过 DCMI_CR 寄存器设置，包括 VSPOL/HSPOL/PCKPOL/数据宽度等重要参数，都在这一步设置，同时我们也开启帧中断，编写 DCMI 中断服务函数，方便进行数据处理（尤其是 JPEG 模式的时候）。不过对于 CAPTURE 位，我们等待 DMA 配置好之后再设置，另外对于 OV2640 输出的 JPEG 数据，我们也不使用 DCMI 的 JPEG 数据模式（实测设置不设置都一样），而是采用正常模式，直接采集。

(4) 配置 DMA。

本实验采用连续模式采集，并将采集到的数据输出到 LCD（RGB565 模式）或内存（JPEG 模式），所以源地址都是 DCMI_DR，而目的地址可能是 TFTLCD->DATA 或者 SRAM 的地址。DCMI 的 DMA 传输采用的是 DMA2 数据流 1 的通道 1 来实现的，关于 DMA 的介绍，请大家参考我们开发板开发攻略教程。

(5) 设置 OV2640 的图像输出大小，使能 DCMI 捕获。

图像输出大小设置，分两种情况：在 RGB565 模式下，我们根据 LCD 的尺寸，设置输出图像大小，以实现全屏显示（图像可能因缩放而变形）；在 JPEG 模式下，我们可以自由设置输出图像大小（可不缩放）；最后，开启 DCMI 捕获，

即可正常工作了。

2 硬件设计

本章实验功能简介：开机后，初始化摄像头模块（OV2640），如果初始化成功，则提示选择模式：RGB565 模式，或者 JPEG 模式。KEY0 用于选择 RGB565 模式，KEY1 用于选择 JPEG 模式。

当使用 RGB565 时，输出图像（固定为：UXGA）将经过缩放处理（完全由 OV2640 的 DSP 控制），显示在 LCD 上面。我们可以通过 KEY_UP 按键选择：1:1 显示，即不缩放，图片不变形，但是显示区域小（液晶分辨率大小），或者缩放显示，即将 1600*1200 的图像压缩到液晶分辨率尺寸显示，图片变形，但是显示了整个图片内容（实测图片变形后显示画面错误，如果有用户调试成功后可反馈给我们改进）。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以设置饱和度；KEY2 按键，可以设置特效。

当使用 JPEG 模式时，图像可以设置任意尺寸（QQVGA~UXGA），采集到的 JPEG 数据将先存放到 STM32 的内存里面，每当采集到一帧数据，就会关闭 DMA 传输，然后将采集到的数据发送到串口 1（此时可以通过上位机软件“\3--软件工具\串口&网络摄像头软件\CAM.exe”接收，并显示图片），之后再重新启动 DMA 传输。我们可以通过 KEY_UP 设置输出图片的尺寸（QQVGA~UXGA）。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以设置饱和度；KEY2 按键，可以设置特效。

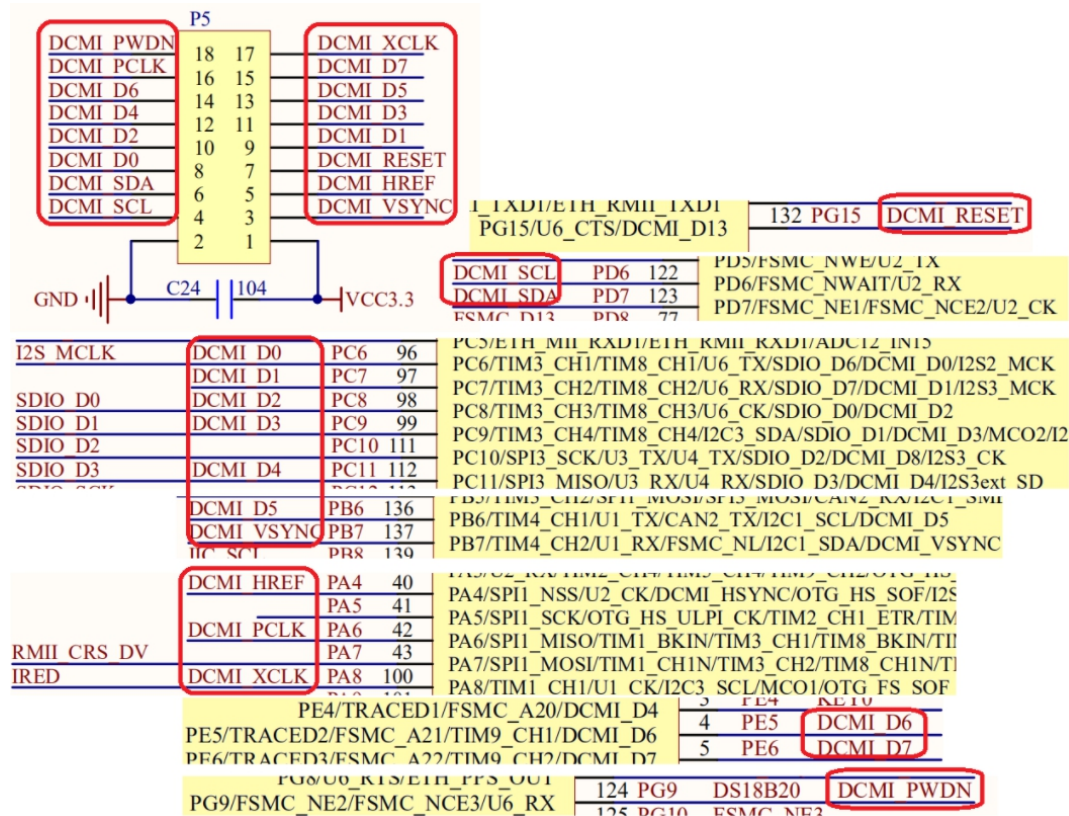
DS0 指示程序运行状态，DS1 用于指示帧中断。

本实验使用到硬件资源如下：

- （1）DS0、DS1 指示灯
- （2）KEY_UP、KEY0、KEY1 和 KEY2 按键
- （3）串口 1
- （4）TFTLCD 模块
- （5）PZ-OV2640 摄像头模块

这些资源，除了最后一个，其他的我们在开发板中开发攻略教程都介绍过。这里我们重点介绍下天马&麒麟开发板的摄像头接口与 PZ-OV2640 摄像头模块

的连接。在开发板的左上角的 2*9 的 P5 排座，是摄像头模块/OLED 模块共用接口。本实验，我们只需要将 PZ-V2640 摄像头模块插入这个接口即可，该接口与 STM32 的连接关系如下图所示：



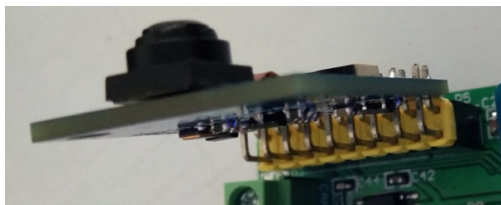
从上图可以看出，OV2640 摄像头模块的各信号脚与 STM32 的连接关系为：

- DCMI_VSYNC 接 PB7;
- DCMI_HREF 接 PA4;
- DCMI_PCLK 接 PA6;
- DCMI_SCL 接 PD6;
- DCMI_SDA 接 PD7;
- DCMI_RESET 接 PG15;
- DCMI_PWDN 接 PG9;
- DCMI_XCLK 接 PA8 (本章未用到);
- DCMI_D[7:0]接 PE6/PE5/PB6/PC11/PC9/PC8/PC7/PC6;

这些线的连接，天马&麒麟开发板的内部已经连接好了，我们只需要将 PZ-OV2640 摄像头模块插上去就好了。板载接口如下：



特别注意：模块插入开发板摄像头接口时注意方向，可对照板载接口背面 PCB 丝印电源脚位即可，PZ-0V2640 插入开发板图如下：



注意：DCMI 摄像头接口和 I2S 接口、DAC、SDIO 以及 DS18B20 等有冲突，使用的时候，必须分时复用才可以，不可同时使用。

3 软件设计

打开本实验工程目录，我们在 APP 文件夹下新建了 OV2640 和 DCMI 两个文件夹。在 ov2640 文件夹下新建了 ov2640.c、sccb.c、ov2640.h、sccb.h、ov2640cfg.h 等 5 个文件，并同时在工程中将这个文件夹加入头文件包含路径。在 dcmi 文件夹新建了 dcmi.c 和 dcmi.h 两个文件，也将这个文件夹加入头文件包含路径。

DCMI 接口相关的库函数分布在 stm32f4xx_dcmi.c 文件以及对应的头文件 stm32f4xx_dcmi.h 中。我们工程引入了这两个文件。

本实验总共新增了 7 个文件，代码比较多，我们就不一一列出了，仅挑几个重要的地方进行讲解。

3.1 ov2640.c

首先，我们来看 ov2640.c 里面的 OV2640_Init 函数，该函数代码如下：

```
1. //初始化 OV2640
2. //配置完以后,默认输出是 1600*1200 尺寸的图片!!
3. //返回值:0,成功
4. // 其他,错误代码
5. u8 OV2640_Init(void)
6. {
```

```

7.      u16 i=0;
8.      u16 reg;
9.      //设置 IO
10.     GPIO_InitTypeDef  GPIO_InitStructure;
11.
12.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOG, ENABLE);
13.     //GPIOG9,15 初始化设置
14.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9|GPIO_Pin_15;//PG9,15 推挽输出
15.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //推挽输出
16.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;//推挽输出
17.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;//100MHz
18.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;//上拉
19.     GPIO_Init(GPIOG, &GPIO_InitStructure);//初始化
20.
21.     OV2640_PWDN=0; //POWER ON
22.     delay_ms(10);
23.     OV2640_RST=0; //复位 OV2640
24.     delay_ms(10);
25.     OV2640_RST=1; //结束复位
26.     SCCB_Init(); //初始化 SCCB 的 IO 口
27.     SCCB_WR_Reg(OV2640_DSP_RA_DLMT, 0x01); //操作 sensor 寄存器
28.     SCCB_WR_Reg(OV2640_SENSOR_COM7, 0x80); //软复位 OV2640
29.     delay_ms(50);
30.     reg=SCCB_RD_Reg(OV2640_SENSOR_MIDH); //读取厂家 ID 高八位
31.     reg<<=8;
32.     reg|=SCCB_RD_Reg(OV2640_SENSOR_MIDL); //读取厂家 ID 低八位
33.     if(reg!=OV2640_MID)
34.     {
35.         printf("MID:%d\r\n",reg);
36.         return 1;
37.     }
38.     reg=SCCB_RD_Reg(OV2640_SENSOR_PIDH); //读取厂家 ID 高八位
39.     reg<<=8;
40.     reg|=SCCB_RD_Reg(OV2640_SENSOR_PIDL); //读取厂家 ID 低八位
41.     if(reg!=OV2640_PID)
42.     {
43.         printf("HID:%d\r\n",reg);
44.         return 2;
45.     }
46.     //初始化 OV2640,采用 SXGA 分辨率(1600*1200)
47.     for(i=0;i<sizeof(ov2640_sxga_init_reg_tbl)/2;i++)
48.     {
49.         SCCB_WR_Reg(ov2640_sxga_init_reg_tbl[i][0],ov2640_sxga_init_reg_tbl[i][1]);
50.     }

```

```
51.     return 0x00;    //ok
52. }
```

此部分代码先初始化 OV2640 相关的 IO 口（包括 SCCB_Init），然后最主要的是完成 OV2640 的寄存器序列初始化。OV2640 的寄存器特多（百几十个），配置特麻烦，幸好厂家有提供参考配置序列（详见《OV2640 Software Application Notes 1.03》），本章我们用到的配置序列，存放在 ov2640_uxga_init_reg_tbl 这个数组里面，该数组是一个 2 维数组，存储初始化序列寄存器及其对应的值，该数组存放在 ov2640cfg.h 里面。

另外，在 ov2640.c 里面，还有几个函数比较重要，这里贴代码了，只介绍功能：

OV2640_Window_Set 函数，该函数用于设置传感器输出窗口；

OV2640_ImageSize_Set 函数，用于设置图像尺寸；

OV2640_ImageWin_Set 函数，用于设置图像窗口大小；

OV2640_OutSize_Set 函数，用于设置图像输出大小；

这就是我们前面小节所介绍的 4 个设置，他们共同决定了图像的输出。

3.2 ov2640cfg.h

接下来，我们看看 ov2640cfg.h 里面 ov2640_uxga_init_reg_tbl 的内容，ov2640cfg.h 文件的代码如下：

```
1.    //OV2640 UXGA 初始化寄存器序列表
2.    //此模式下帧率为 15 帧
3.    //UXGA(1600*1200)
4.    const u8 ov2640_uxga_init_reg_tbl[][2]=
5.    {
6.        0xff, 0x00,
7.        .....//省略部分代码
8.        0x05, 0x00,
9.    };
10.   //OV2640 SVGA 初始化寄存器序列表
11.   //此模式下,帧率可以达到 30 帧
12.   //SVGA 800*600
13.   const u8 ov2640_svga_init_reg_tbl[][2]=
14.   {
15.       0xff, 0x00,
16.       .....//省略部分代码
```

```

17.     0x05, 0x00,
18. };
19. const u8 ov2640_yuv422_reg_tbl[][2]=
20. {
21.     0xFF, 0x00,
22.     .....//省略部分代码
23.     0x00, 0x00,
24. };
25. const u8 ov2640_jpeg_reg_tbl[][2]=
26. {
27.     0xff, 0x01,
28.     .....//省略部分代码
29.     0xe0, 0x00,
30. };
31. const u8 ov2640_rgb565_reg_tbl[][2]=
32. {
33.     0xFF, 0x00,
34.     .....//省略部分代码
35.     0xe0, 0x00,
36. };

```

以上代码，我们省略了很多（全部贴出来太长了），里面总共有 5 个数组。我们大概了解下数组结构，每个数组条目的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如{0xff, 0x01}，就表示在 0Xff 地址，写入 0X01 这个值。

五个数组里面 ov2640_uxga_init_reg_tbl 和 ov2640_svga_init_reg_tbl，分别用于配置 OV2640 输出 UXGA 和 SVGA 分辨率的图像，我们只用了 ov2640_uxga_init_reg_tbl 这个数组，完成对 OV2640 的初始化(设置为 UXGA)。最后 OV2640 要输出数据是 RGB565 还是 JPEG,就得通过其他数组设置，输出 RGB565 时，通过一个数组：ov2640_rgb565_reg_tbl 设置即可；输出 JPEG 时，则要通过 ov2640_yuv422_reg_tbl 和 ov2640_jpeg_reg_tbl 两个数组设置。

3.3 dcmi.c

接下来，我们看看 dcmi.c 里面的代码，如下：

```

1.  #include "dcmi.h"
2.  #include "tftlcd.h"
3.  #include "led.h"

```

```

4.  #include "ov2640.h"
5.
6.
7.  u8 ov_frame=0;                //帧率
8.  extern void jpeg_data_process(void); //JPEG 数据处理函数
9.
10. DCMI_InitTypeDef DCMI_InitStructure;
11.
12. //DCMI 中断服务函数
13. void DCMI_IRQHandler(void)
14. {
15.     if(DCMI_GetITStatus(DCMI_IT_FRAME)==SET)//捕获到一帧图像
16.     {
17.         jpeg_data_process(); //jpeg 数据处理
18.         DCMI_ClearITPendingBit(DCMI_IT_FRAME);//清除帧中断
19.         LED2=!LED2;
20.         ov_frame++;
21.     }
22. }
23. //DCMI DMA 配置
24. //DMA_Memory0BaseAddr:存储器地址 将要存储摄像头数据的内存地址(也可以是外设地址)
25. //DMA_BufferSize:存储器长度 0~65535
26. //DMA_MemoryDataSize:存储器位宽
27. //DMA_MemoryDataSize: 存 储 器 位 宽
   @defgroup DMA_memory_data_size :DMA_MemoryDataSize_Byte/DMA_MemoryDataSize_HalfWord/DMA_
   MemoryDataSize_Word
28. //DMA_MemoryInc: 存 储 器 增 长 方 式
   @defgroup DMA_memory_incremented_mode /** @defgroup DMA_memory_incremented_mode : DMA_Mem
   oryInc_Enable/DMA_MemoryInc_Disable
29. void DCMI_DMA_Init(u32 DMA_Memory0BaseAddr,u16 DMA_BufferSize,u32 DMA_MemoryDataSize,u32 DM
   A_MemoryInc)
30. {
31.     DMA_InitTypeDef DMA_InitStructure;
32.
33.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2,ENABLE);//DMA2 时钟使能
34.     DMA_DeInit(DMA2_Stream1);
35.     while (DMA_GetCmdStatus(DMA2_Stream1) != DISABLE){} //等待 DMA2_Stream1 可配置
36.
37.     /* 配置 DMA Stream */
38.     DMA_InitStructure.DMA_Channel = DMA_Channel_1; //通道 1 DCMI 通道
39.     DMA_InitStructure.DMA_PeripheralBaseAddr = (u32)&DCMI->DR;//外设地址为:DCMI->DR
40.     DMA_InitStructure.DMA_Memory0BaseAddr = DMA_Memory0BaseAddr;//DMA 存储器 0 地址
41.     DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralToMemory;//外设到存储器模式
42.     DMA_InitStructure.DMA_BufferSize = DMA_BufferSize;//数据传输量

```

```

43.     DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable; //外设非增量模式
44.     DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc; //存储器增量模式
45.     DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word; //外设数据长度:32
    位
46.     DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize; //存储器数据长度
47.     DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; // 使用循环模式
48.     DMA_InitStructure.DMA_Priority = DMA_Priority_High; //高优先级
49.     DMA_InitStructure.DMA_FIFOMode = DMA_FIFOMode_Enable; //FIFO 模式
50.     DMA_InitStructure.DMA_FIFOThreshold = DMA_FIFOThreshold_Full; //使用全 FIFO
51.     DMA_InitStructure.DMA_MemoryBurst = DMA_MemoryBurst_Single; //外设突发单次传输
52.     DMA_InitStructure.DMA_PeripheralBurst = DMA_PeripheralBurst_Single; //存储器突发单次传输
53.     DMA_Init(DMA2_Stream1, &DMA_InitStructure); //初始化 DMA Stream
54.
55. }
56. //DCMI 初始化
57. void My_DCMI_Init(void)
58. {
59.     GPIO_InitTypeDef GPIO_InitStructure;
60.     NVIC_InitTypeDef NVIC_InitStructure;
61.
62.
63.     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA|RCC_AHB1Periph_GPIOB|RCC_AHB1Periph_GPIOC|R
        CC_AHB1Periph_GPIOE, ENABLE); //使能 GPIOA B C E 时钟
64.     RCC_AHB2PeriphClockCmd(RCC_AHB2Periph_DCMI, ENABLE); //使能 DCMI 时钟
65.     //PA4/6 初始化设置
66.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4|GPIO_Pin_6; //PA4/6 复用功能输出
67.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; //复用功能输出
68.     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽输出
69.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz; //100MHz
70.     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉
71.     GPIO_Init(GPIOA, &GPIO_InitStructure); //初始化
72.
73.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_7|GPIO_Pin_6; // PB6/7 复用功能输出
74.     GPIO_Init(GPIOB, &GPIO_InitStructure); //初始化
75.
76.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6|GPIO_Pin_7|GPIO_Pin_8|GPIO_Pin_9|GPIO_Pin_11; //
        /PC6/7/8/9/11 复用功能输出
77.     GPIO_Init(GPIOC, &GPIO_InitStructure); //初始化
78.
79.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5|GPIO_Pin_6; //PE5/6 复用功能输出
80.     GPIO_Init(GPIOE, &GPIO_InitStructure); //初始化
81.
82.     GPIO_PinAFConfig(GPIOA, GPIO_PinSource4, GPIO_AF_DCMI); //PA4, AF13 DCMI_HSYNC
83.     GPIO_PinAFConfig(GPIOA, GPIO_PinSource6, GPIO_AF_DCMI); //PA6, AF13 DCMI_PCLK

```

```

84.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource7,GPIO_AF_DCMI); //PB7,AF13  DCMI_VSYNC
85.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource6,GPIO_AF_DCMI); //PC6,AF13  DCMI_D0
86.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource7,GPIO_AF_DCMI); //PC7,AF13  DCMI_D1
87.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource8,GPIO_AF_DCMI); //PC8,AF13  DCMI_D2
88.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource9,GPIO_AF_DCMI); //PC9,AF13  DCMI_D3
89.     GPIO_PinAFConfig(GPIOC,GPIO_PinSource11,GPIO_AF_DCMI); //PC11,AF13  DCMI_D4
90.     GPIO_PinAFConfig(GPIOB,GPIO_PinSource6,GPIO_AF_DCMI); //PB6,AF13  DCMI_D5
91.     GPIO_PinAFConfig(GPIOE,GPIO_PinSource5,GPIO_AF_DCMI); //PE5,AF13  DCMI_D6
92.     GPIO_PinAFConfig(GPIOE,GPIO_PinSource6,GPIO_AF_DCMI); //PE6,AF13  DCMI_D7
93.
94.     DCMI_DeInit();//清除原来的设置
95.
96.     DCMI_InitStructure.DCMI_CaptureMode=DCMI_CaptureMode_Continuous;//连续模式
97.     DCMI_InitStructure.DCMI_CaptureRate=DCMI_CaptureRate_All_Frame;//全帧捕获
98.     DCMI_InitStructure.DCMI_ExtendedDataMode= DCMI_ExtendedDataMode_8b;//8 位数据格式
99.     DCMI_InitStructure.DCMI_HSPolarity= DCMI_HSPolarity_Low;//HSYNC 低电平有效
100.    DCMI_InitStructure.DCMI_PCKPolarity= DCMI_PCKPolarity_Rising;//PCLK 上升沿有效
101.    DCMI_InitStructure.DCMI_SynchroMode= DCMI_SynchroMode_Hardware;//硬件同步 HSYNC,VSYNC
102.    DCMI_InitStructure.DCMI_VSPolarity=DCMI_VSPolarity_Low;//VSYNC 低电平有效
103.    DCMI_Init(&DCMI_InitStructure);
104.
105.    DCMI_ITConfig(DCMI_IT_FRAME,ENABLE);//开启帧中断
106.
107.    DCMI_Cmd(ENABLE);    //DCMI 使能
108.
109.    NVIC_InitStructure.NVIC_IRQChannel = DCMI_IRQn;
110.    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=1;//抢占优先级 1
111.    NVIC_InitStructure.NVIC_IRQChannelSubPriority =2;    //子优先级 3
112.    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;    //IRQ 通道使能
113.    NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器、
114.
115. }
116. //DCMI,启动传输
117. void DCMI_Start(void)
118. {
119.     //对于一些型号彩屏，需要修改下窗口，要么-1，要么不减 1
120.     LCD_Set_Window(0, 0, tftlcd_data.width-1, tftlcd_data.height-1); //设置点的位置
121.     DMA_Cmd(DMA2_Stream1, ENABLE);//开启 DMA2,Stream1
122.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
123. }
124. //DCMI,关闭传输
125. void DCMI_Stop(void)
126. {
127.     DCMI_CaptureCmd(DISABLE);//DCMI 捕获使关闭

```

```

128.
129.     while(DCMI->CR&0X01);           //等待传输结束
130.
131.     DMA_Cmd(DMA2_Stream1,DISABLE);//关闭 DMA2,Stream1
132. }
133. ///////////////////////////////////////////////////////////////////
134. //以下两个函数,供 usmart 调用,用于调试代码
135.
136. //DCMI 设置显示窗口
137. //sx,sy;LCD 的起始坐标
138. //width,height:LCD 显示范围.
139. void DCMI_Set_Window(u16 sx,u16 sy,u16 width,u16 height)
140. {
141.     DCMI_Stop();
142.     LCD_Clear(WHITE);
143.     LCD_Set_Window(sx,sy,width,height);
144.     OV2640_OutSize_Set(width,height);
145.
146.     DMA_Cmd(DMA2_Stream1,ENABLE);    //开启 DMA2,Stream1
147.
148.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
149.
150. }
151.
152. //通过 usmart 调试,辅助测试用.
153. //pclk/hsync/vsync:三个信号的有限电平设置
154. void DCMI_CR_Set(u8 pclk,u8 hsync,u8 vsync)
155. {
156.     DCMI_DeInit();//清除原来的设置
157.
158.     DCMI_InitStructure.DCMI_CaptureMode=DCMI_CaptureMode_Continuous;//连续模式
159.     DCMI_InitStructure.DCMI_CaptureRate=DCMI_CaptureRate_All_Frame;//全帧捕获
160.     DCMI_InitStructure.DCMI_ExtendedDataMode= DCMI_ExtendedDataMode_8b;//8 位数据格式
161.     DCMI_InitStructure.DCMI_HSPolarity= hsync<<6;//HSYNC 低电平有效
162.     DCMI_InitStructure.DCMI_PCKPolarity= pclk<<5;//PCLK 上升沿有效
163.     DCMI_InitStructure.DCMI_SynchroMode= DCMI_SynchroMode_Hardware;//硬件同步 HSYNC,VSYNC
164.     DCMI_InitStructure.DCMI_VSPolarity=vsync<<7;//VSYNC 低电平有效
165.     DCMI_Init(&DCMI_InitStructure);
166.
167.     DCMI_CaptureCmd(ENABLE);//DCMI 捕获使能
168.     DCMI_Cmd(ENABLE);    //DCMI 使能
169. }

```

其中：DCMI_IRQHandler 函数，用于处理帧中断，可以实现帧率统计（需要

定时器支持)和 JPEG 数据处理等。DCMI_DMA_Init 函数,则用于配置 DCMI 的 DMA 传输,其外设地址固定为:DCMI->DR,而存储器地址可变(LCD 或者 SRAM)。DMA 被配置为循环模式,一旦开启,DMA 将不停的循环传输数据。My_DCMI_Init 函数用于初始化 STM32F4 的 DCMI 接口,这是根据前面小节提到的配置步骤进行配置的。最后,DCMI_Start 和 DCMI_Stop 两个函数,用于开启或停止 DCMI 接口。

其他部分代码我们就不再细说了,请大家参考实验例程。

3.4 main.c

最后,打开 main.c 文件,代码如下:

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "usart.h"
4.  #include "led.h"
5.  #include "key.h"
6.  #include "tftlcd.h"
7.  #include "time.h"
8.  #include "ov2640.h"
9.  #include "dcmi.h"
10.
11.
12.  u8 ov2640_mode=0;           //工作模式:0,RGB565 模式;1,JPEG 模式
13.
14.  #define jpeg_buf_size 31*1024 //定义 JPEG 数据缓存 jpeg_buf 的大小(*4 字节)
15.  __align(4) u32 jpeg_buf[jpeg_buf_size]; //JPEG 数据缓存 buf
16.  volatile u32 jpeg_data_len=0; //buf 中的 JPEG 有效数据长度
17.  volatile u8 jpeg_data_ok=0; //JPEG 数据采集完成标志
18.                                //0,数据没有采集完;
19.                                //1,数据采集完了,但是还没处理;
20.                                //2,数据已经处理完成了,可以开始下一帧接收
21.  //JPEG 尺寸支持列表
22.  const u16 jpeg_img_size_tbl[][2]=
23.  {
24.      176,144, //QCIF
25.      160,120, //QQVGA
26.      352,288, //CIF
27.      320,240, //QVGA
28.      640,480, //VGA
29.      800,600, //SVGA
```

```

30.     1024,768,    //XGA
31.     1280,1024,  //SXGA
32.     1600,1200,  //UXGA
33. };
34.     const u8*EFFECTS_TBL[7]={ "Normal", "Negative", "B&W", "Redish", "Greenish", "Bluish", "Antique"};
    //7 种特效
35.     const u8*JPEG_SIZE_TBL[9]={ "QCIF", "QQVGA", "CIF", "QVGA", "VGA", "SVGA", "XGA", "SXGA", "UXGA"};
    //JPEG 图片 9 种尺寸
36.
37.
38.     //处理 JPEG 数据
39.     //当采集完一帧 JPEG 数据后,调用此函数,切换 JPEG BUF.开始下一帧采集.
40.     void jpeg_data_process(void)
41.     {
42.         if(ov2640_mode)//只有在 JPEG 格式下,才需要做处理.
43.         {
44.             if(jpeg_data_ok==0) //jpeg 数据还未采集完?
45.             {
46.                 DMA_Cmd(DMA2_Stream1, DISABLE); //停止当前传输
47.                 while (DMA_GetCmdStatus(DMA2_Stream1) != DISABLE){} //等待 DMA2_Stream1 可配置
48.                 jpeg_data_len=jpeg_buf_size-DMA_GetCurrDataCounter(DMA2_Stream1); //得到此次数据传
                输的长度
49.
50.                 jpeg_data_ok=1; //标记 JPEG 数据采集完按成,等待其他函数处理
51.             }
52.             if(jpeg_data_ok==2) //上一次的 jpeg 数据已经被处理了
53.             {
54.                 DMA2_Stream1->NDTR=jpeg_buf_size;
55.                 DMA_SetCurrDataCounter(DMA2_Stream1,jpeg_buf_size); //传输长度为 jpeg_buf_size*4 字
                节
56.                 DMA_Cmd(DMA2_Stream1, ENABLE); //重新传输
57.                 jpeg_data_ok=0; //标记数据未采集
58.             }
59.         }
60.     }
61.     //JPEG 测试
62.     //JPEG 数据,通过串口 1 发送给电脑.
63.     void jpeg_test(void)
64.     {
65.         u32 i;
66.         u8 *p;
67.         u8 key;
68.         u8 effect=0,saturation=2,contrast=2;
69.         u8 size=3; //默认是 QVGA 320*240 尺寸

```

```

70.     u8 msgbuf[15]; //消息缓存区
71.     LCD_Clear(WHITE);
72.     FRONT_COLOR=RED;
73.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
74.     LCD_ShowString(30,70,200,16,16,"OV2640 JPEG Mode");
75.     LCD_ShowString(30,100,200,16,16,"KEY0:Contrast"); //对比度
76.     LCD_ShowString(30,120,200,16,16,"KEY1:Saturation"); //色彩饱和度
77.     LCD_ShowString(30,140,200,16,16,"KEY2:Effects"); //特效
78.     LCD_ShowString(30,160,200,16,16,"KEY_UP:Size"); //分辨率设置
79.     sprintf((char*)msgbuf,"JPEG Size:%s",JPEG_SIZE_TBL[size]);
80.     LCD_ShowString(30,180,200,16,16,msgbuf); //显示当前 JPEG 分辨率
81.
82.     OV2640_JPEG_Mode(); //JPEG 模式
83.     My_DCMI_Init(); //DCMI 配置
84.     DCMI_DMA_Init((u32)&jpeg_buf,jpeg_buf_size,DMA_MemoryDataSize_Word,DMA_MemoryInc_Enable)
        ;//DCMI DMA 配置
85.     OV2640_OutSize_Set(jpeg_img_size_tbl[size][0],jpeg_img_size_tbl[size][1]);//设置输出尺
        寸
86.     DCMI_Start(); //启动传输
87.     while(1)
88.     {
89.         if(jpeg_data_ok==1) //已经采集完一帧图像了
90.         {
91.             p=(u8*)jpeg_buf;
92.             LCD_ShowString(30,210,210,16,16,"Sending JPEG data..."); //提示正在传输数据
93.             for(i=0;i<jpeg_data_len*4;i++) //dma 传输 1 次等于 4 字节,所以乘以 4.
94.             {
95.                 while(USART_GetFlagStatus(USART1,USART_FLAG_TC)==RESET); //循环发送,直到发
                    送完毕
96.                 USART_SendData(USART1,p[i]);
97.                 key=KEY_Scan(0);
98.                 if(key)break;
99.             }
100.            if(key) //有按键按下,需要处理
101.            {
102.                LCD_ShowString(30,210,210,16,16,"Quit Sending data "); //提示退出数据传输
103.                switch(key)
104.                {
105.                    case KEY0_PRESS: //对比度设置
106.                        contrast++;
107.                        if(contrast>4)contrast=0;
108.                        OV2640_Contrast(contrast);
109.                        sprintf((char*)msgbuf,"Contrast:%d",(signed char)contrast-2);
110.                        break;

```

```

111.             case KEY1_PRESS:    //饱和度 Saturation
112.                 saturation++;
113.                 if(saturation>4)saturation=0;
114.                 OV2640_Color_Saturation(saturation);
115.                 sprintf((char*)msgbuf, "Saturation:%d", (signed char)saturation-2);
116.                 break;
117.             case KEY2_PRESS:    //特效设置
118.                 effect++;
119.                 if(effect>6)effect=0;
120.                 OV2640_Special_Effects(effect); //设置特效
121.                 sprintf((char*)msgbuf, "%s", EFFECTS_TBL[effect]);
122.                 break;
123.             case KEY_UP_PRESS:  //JPEG 输出尺寸设置
124.                 size++;
125.                 if(size>8)size=0;
126.                 OV2640_OutSize_Set(jpeg_img_size_tbl[size][0], jpeg_img_size_tbl[size][1]); //设置输出尺寸
127.                 sprintf((char*)msgbuf, "JPEG Size:%s", JPEG_SIZE_TBL[size]);
128.                 break;
129.         }
130.         LCD_Fill(30, 180, 239, 190+16, WHITE);
131.         LCD_ShowString(30, 180, 210, 16, 16, msgbuf); //显示提示内容
132.         delay_ms(800);
133.     }else LCD_ShowString(30, 210, 210, 16, 16, "Send data complete!"); //提示传输结束设置
134.     jpeg_data_ok=2; //标记 jpeg 数据处理完了, 可以让 DMA 去采集下一帧了.
135. }
136. }
137. }
138. //RGB565 测试
139. //RGB 数据直接显示在 LCD 上面
140. void rgb565_test(void)
141. {
142.     u8 key;
143.     u8 effect=0, saturation=2, contrast=2;
144.     u8 scale=1;    //默认是全尺寸缩放
145.     u8 msgbuf[15]; //消息缓存区
146.     LCD_Clear(WHITE);
147.     FRONT_COLOR=RED;
148.     LCD_ShowString(30, 50, 200, 16, 16, "PRECHIN-STM32");
149.     LCD_ShowString(30, 70, 200, 16, 16, "OV2640 RGB565 Mode");
150.
151.     LCD_ShowString(30, 100, 200, 16, 16, "KEY0:Contrast"); //对比度
152.     LCD_ShowString(30, 130, 200, 16, 16, "KEY1:Saturation"); //色彩饱和度

```

```

153.     LCD_ShowString(30,150,200,16,16,"KEY2:Effects");           //特效
154.     LCD_ShowString(30,170,200,16,16,"KEY_UP:FullSize/Scale");   //1:1 尺寸(显示真实尺寸)/全尺
    寸缩放
155.
156.     OV2640_RGB565_Mode();    //RGB565 模式
157.     My_DCMI_Init();          //DCMI 配置
158.     DCMI_DMA_Init((u32)&TFTLCD->LCD_DATA,1,DMA_MemoryDataSize_HalfWord,DMA_MemoryInc_Disabl
    e);//DCMI DMA 配置
159.     OV2640_OutSize_Set(tftlcd_data.width,tftlcd_data.height);
160.     DCMI_Start();            //启动传输
161.     while(1)
162.     {
163.         key=KEY_Scan(0);
164.         if(key)
165.         {
166.             DCMI_Stop(); //停止显示
167.             switch(key)
168.             {
169.                 case KEY0_PRESS:    //对比度设置
170.                     contrast++;
171.                     if(contrast>4)contrast=0;
172.                     OV2640_Contrast(contrast);
173.                     sprintf((char*)msgbuf,"Contrast:%d",(signed char)contrast-2);
174.                     break;
175.                 case KEY1_PRESS:    //饱和度 Saturation
176.                     saturation++;
177.                     if(saturation>4)saturation=0;
178.                     OV2640_Color_Saturation(saturation);
179.                     sprintf((char*)msgbuf,"Saturation:%d",(signed char)saturation-2);
180.                     break;
181.                 case KEY2_PRESS:    //特效设置
182.                     effect++;
183.                     if(effect>6)effect=0;
184.                     OV2640_Special_Effects(effect);//设置特效
185.                     sprintf((char*)msgbuf,"%s",EFFECTS_TBL[effect]);
186.                     break;
187.                 case KEY_UP_PRESS:  //1:1 尺寸(显示真实尺寸)/缩放
188.                     scale=!scale;
189.                     if(scale==0)
190.                     {
191.                         OV2640_ImageWin_Set((1600-tftlcd_data.width)/2,(1200-tftlcd_data.he
    ight)/2,tftlcd_data.width,tftlcd_data.height);//1:1 真实尺寸
192.                         OV2640_OutSize_Set(tftlcd_data.width,tftlcd_data.height);
193.                         sprintf((char*)msgbuf,"Full Size 1:1");

```

```

194.         }else
195.         {
196.             OV2640_ImageWin_Set(0,0,1600,1200);           //全尺寸缩放
197.             OV2640_OutSize_Set(tftlcd_data.width,tftlcd_data.height);
198.             sprintf((char*)msgbuf,"Scale");
199.         }
200.         break;
201.     }
202.     LCD_ShowString(30,50,210,16,16,msgbuf);//显示提示内容
203.     delay_ms(800);
204.     DCMI_Start();//重新开始传输
205. }
206.     delay_ms(10);
207. }
208. }
209. int main(void)
210. {
211.     u8 key;
212.     u8 t;
213.
214.     SysTick_Init(168); //初始化延时函数
215.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//设置系统中断优先级分组 2
216.     USART1_Init(921600); //初始化串口波特率为 921600
217.     LED_Init(); //初始化 LED
218.     TFTLCD_Init(); //LCD 初始化
219.     KEY_Init(); //按键初始化
220.     TIM3_Init(10000-1,8400-1);//10Khz 计数,1 秒钟中断一次
221.
222.     FRONT_COLOR=RED;//设置字体为红色
223.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
224.     LCD_ShowString(30,70,200,16,16,"OV2640 TEST");
225.     LCD_ShowString(30,90,200,16,16,"www.prechin.net");
226.     while(OV2640_Init())//初始化 OV2640
227.     {
228.         LCD_ShowString(30,130,240,16,16,"OV2640 ERR");
229.         delay_ms(200);
230.         LCD_Fill(30,130,239,170,WHITE);
231.         delay_ms(200);
232.     }
233.     LCD_ShowString(30,130,200,16,16,"OV2640 OK");
234.     while(1)
235.     {
236.         key=KEY_Scan(0);
237.         if(key==KEY0_PRESS) //RGB565 模式

```

```

238.     {
239.         ov2640_mode=0;
240.         break;
241.     }else if(key==KEY1_PRESS)    //JPEG 模式
242.     {
243.         ov2640_mode=1;
244.         break;
245.     }
246.     t++;
247.     if(t==100)
248.     {
249.         LED1=!LED1;
250.         LCD_ShowString(30,150,230,16,16,"KEY0:RGB565  KEY1:JPEG"); //闪烁显示提示信息
251.     }
252.     if(t==200)
253.     {
254.         LCD_Fill(30,150,210,150+16,WHITE);
255.         t=0;
256.     }
257.     delay_ms(5);
258. }
259. if(ov2640_mode)jpeg_test();
260. else rgb565_test();
261. }

```

这里定义了一个非常大的数组 jpeg_buf（124KB），用来存储 JPEG 数据，因为 1600*1200 大小的 jpeg 图片，有可能大于 120K，所以必须将这个数组尽量设置大一点。

在 main.c 里面，总共有 4 个函数：jpeg_data_process、jpeg_test、rgb565_test 和 main 函数。其中，jpeg_data_process 函数用于处理 JPEG 数据的接收，在 DCMI_IRQHandler 函数里面被调用，通过一个 jpeg_data_ok 变量与 jpeg_test 函数共同控制 JPEG 的数据传送。jpeg_test 函数将 OV2640 设置为 JPEG 模式，该函数接收 OV2640 的 JPEG 数据，并通过串口 1 发送给上位机。rgb565_test 函数将 OV2640 设置为 RGB565 模式，并将接收到的数据，直接传送给 LCD，处理过程完全由硬件实现，CPU 完全不用理会。最后，main 函数就相对简单了，大家看代码即可。

4 实验现象

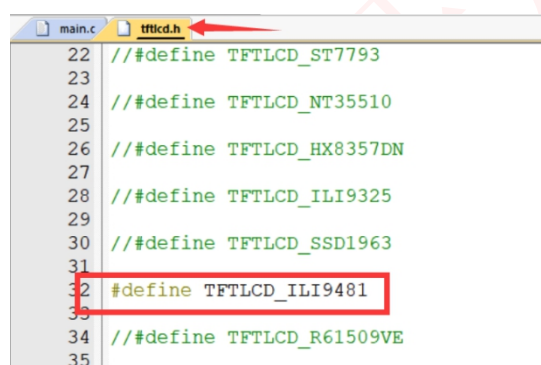
首先，请先确保硬件都已经连接好：

①PZ-OV2640 模块与天马&麒麟开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口摄像头工具显示图像。

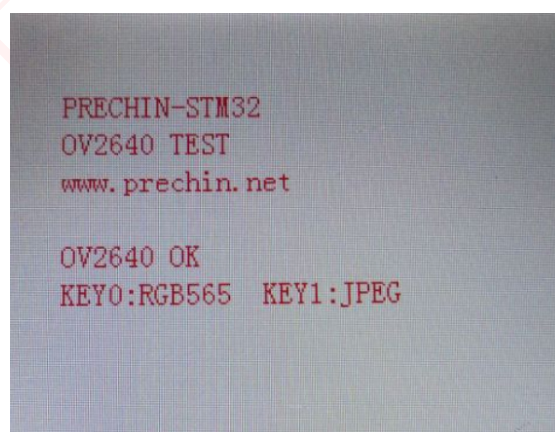
③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以麒麟开发板为展示）LCD 界面显示如下图所示：



在 OV2640 初始化成功后，屏幕提示选择模式，此时我们可以按 KEY0，进入 RGB565 模式测试，也可以按 KEY1，进入 JPEG 模式测试。

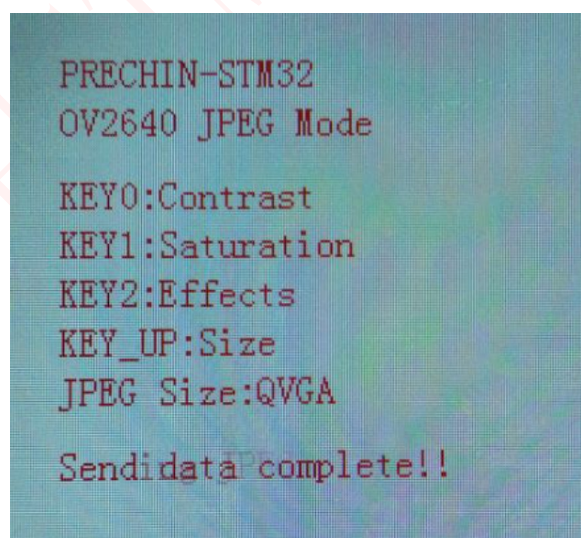
当按 KEY0 后，选择 RGB565 模式，LCD 满屏显示压缩放后的图像，如下图

所示：



同时还可以通过 KEY0 按键，设置对比度；KEY1 按键，设置饱和度；KEY2，可以设置特效。

当按 KEY1 后，选择 JPEG 模式，此时屏幕显示 JPEG 数据传输进程，如下图所示：



默认条件下，图像分辨率是 QVGA(320*240)的，此时通过串口 1 发送图像数据到上位机。我们打开上位机软件“\3--软件工具\串口&网络摄像头软件\CAM.exe”，选择正确的串口，即开发板上 CH340 串口，然后波特率设置为 921600，打开即可收到 STM32 传过来的图片了，如下图所示：



我们可以通过 KEY_UP 设置输出图像的尺寸（QQVGA~UXGA）。通过 KEY0 按键，设置对比度；KEY1 按键，设置饱和度；KEY2 按键，设置特效。

在 time.c 的 TIM3 中断函数中，我们把帧率输出注释掉了，如果要查看多少帧，可以自行打开。

5 其他

（1）购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

（2）资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

（3）技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）